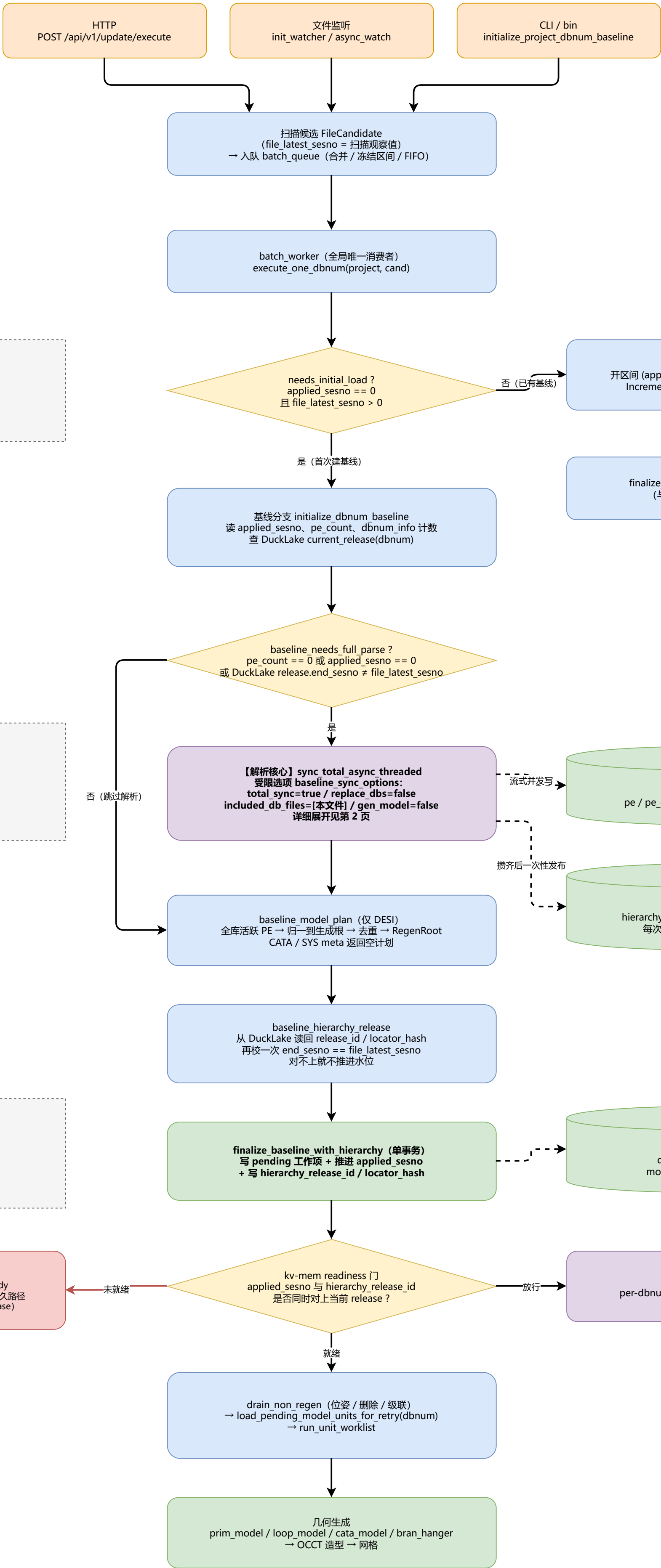


aios-database 数据解析总流程：从触发到模型生成



applied_sesno = 应用水位 (权威值)
「我真的处理完并收口到这里了」

file_latest_sesno = 扫描观察值
「我刚才看了一眼，文件长到这」
两者混用 = 把没处理的当处理过了

解析本身幂等 (replace_dbs=false)
所以「白跑一遍」只损失时间，
不损失正确性。

但不要手动开 total_sync:
sync_pdms 不写水位，独立跑完
基线路径还会再解析一遍。

必须先收口再生成：
生成依赖 current_hierarchy 的当前态
查询，而它要等 release 写进水位后
readiness 门才放行。

数据成功、模型失败不回滚数据。

解析核心：一次解析，两条互不相干的写路径

共用解析主链

入口 sync_total_async_threaded
(db_option, project, dbno_set, db_types, progress)

遍历 children_files (逐个 db 文件)
读 60 字节文件头 parse_file_basic_info → db_type / db_no
过滤: db_types 命中 + included_db_files
(SYS 元数据不受 included_db_files 约束, ADR-007)

PdmsIO::open → get_latest_sesno
sesno == 0 直接 bail (没有可发布的 session)
io.open() 顺带构建 ses_range_map

parse file db basic data → db basic
refno table_map / children_map / world_refno
解析失败绝不退化成空库, 整批 baseline 失败

SurrealDB 生产者

save_pe_relates(db_basic)
→ SenderJsonsData::PERelateJson

门 A · 层级预计算 (写盘前)
1) is_projected_hierarchy noun 过滤, 挡掉 MNUM 等数据库控制记录
2) children_map → sibling_orders; 子节点重复出现 / 父不是元素 → 失败
3) required_hierarchy_refnos = 全部层级子节点 + world_refno
4) hierarchy_ref0s 从 refno_table_map 取 (排除 0 与哨兵)

chunk 循环 all_refnos.chunks(sync_chunk_size)
parse_file_with_chunk(db_basic, chunk, ses_range_map,
ignore_world_refno)
→ total_attr_map / type_ele_map
(WORL 只在持有它的那一块解析一次)

save_pes(db_basic, total_attr_map, db_no, ...)
→ SenderJsonsData::PEJson

按 noun 分表逐条生成 JSON
att.gen_sur_json() → 属性表
att.gen_sur_json_uda() → normalize_sql_string → ATT_UDA
→ SenderJsonsData::AttJson((表名, jsons))

DuckLake 层级投影

HierarchyRowV1::from_named_attr_map
refno 一致性校验 (map vs attr)
validate_member_owner: 成员表里的父
与标量 OWNER 不一致 → 直接失败

hierarchy_rows 内存累加 (不立即写盘)
只有 6 个字段: dbnum / refno / owner / noun / name / sibling_order

门 B · 完整性 (单文件解析结束后)
parsed_refnos 必须覆盖 required_hierarchy_refnos
缺一个就整库失败, 错误里带 parsed / required / missing_sample

门 C · BaselineProjection::new_with_ref0s
ref0 为 0 或哨兵 → 失败; ref0 不在文件索引内 → 失败
row.dbnum 不符 / refno 重复 → 失败
validate_hierarchy_rows → 按 refno 排序 → 推入 hierarchy_baselines

本页是第 1 页【解析核心】的展开。
基线路径用受限选项调用:
total_sync=true / replace_dbs=false
included_db_files=[本文件]
manual_db_nums=[本 dbnum]
gen_model=false / gen_mesh=false
所以只解析这一个库, 且不顺带生成。

SurrealDB 写入管线

flume::bounded(BASELINE_QUEUE_CAPACITY)
背压: 无界队列会把未落盘的 INSERT
payload 全留住, 撑爆进程

写入 worker × BASELINE_WRITE_WORKERS
chunks(BASELINE_WRITE_WINDOW)
execute_surreal_checked + 冲突重试
(乐观事务下无界并发会导致静默丢写)

SurrealDB
INSERT IGNORE INTO pe
INSERT RELATION INTO pe_owner
INSERT IGNORE INTO (noun) / ATT_UDA

两条写路径彼此独立, 不在同一个事务里:
· SurrealDB 边解析边流式并发写
· DuckLake 全部攒在内存, 等 SurrealDB
写管线收尾后才一次性发布
崩溃窗口 (PE 已落, DuckLake 未提交)
靠水位与 readiness 门兜底, 不靠事务。

finish_write_pipeline(sender, insert_handles, parser_outcome)
关闭 sender → drain 全部 writer → 汇总解析错误与写入错误
(解析失败也要先把 writer 排干, 再上抛)

spawn_blocking: HierarchyProjectionStore::publish_baseline
幕等短路: end_sesno + source_digest + change_hash 全等 →
already_committed
release_id = hash(project, dbnum, base_release_id, 0, end_sesno, digest,
change_hash)

prepare_locator + stage_locator
内容寻址 ref0 → dbnum 反查表
先留存, 不激活

DuckLake 事务 (DuckDB ducklake 扩展, SHA256 钉版)
BEGIN TRANSACTION
→ DELETE FROM hierarchy_node WHERE dbnum = ?
→ appender_to_catalog_and_db 逐行追加
→ INSERT INTO hierarchy_release(...)
→ CALL set_commit_message('aios', release_id, extra_info =>
locator_hash)
→ COMMIT (失败则 ROLLBACK)

DuckLake
hierarchy_node (按 dbnum 分区)
hierarchy_release + snapshot

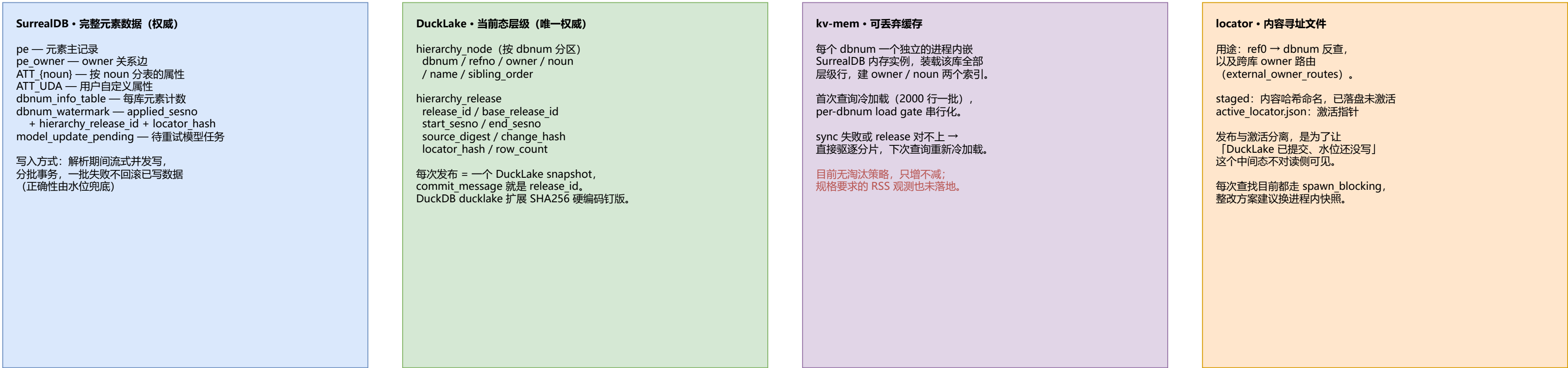
last_committed_snapshot() → snapshot_id
返回 ProjectionCommit(release_id, locator_hash, snapshot_id, row_count)

evict_global_if_resident(project, dbnum)
基线换了 → 立刻丢弃可能常驻的 kv-mem 分片

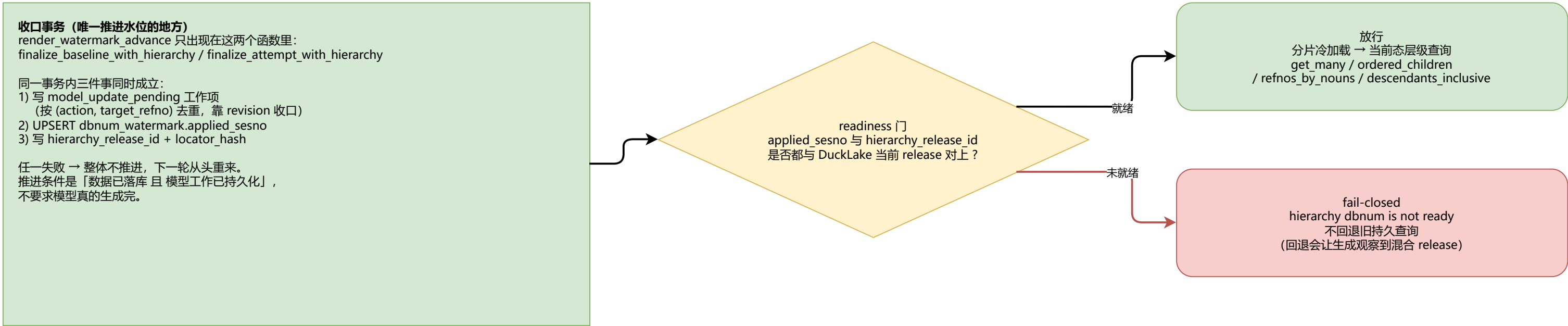
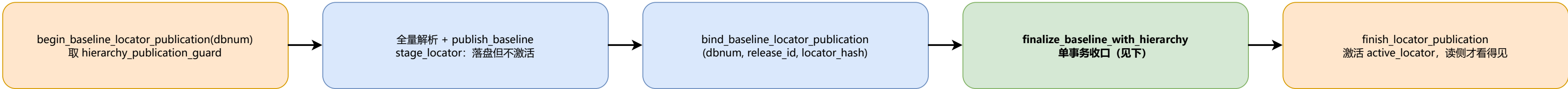
define_owner_index / define_pe_index
rebuild_dbnum_info_from_pe (仅 total_sync)

返回 parsed_db_infos (dbnum → 元素数)
交回 initialize_dbnum_baseline 做收口 (第 1 页)

存储职责边界、locator 三段发布协议与就绪门



locator 三段发布协议（仅在需要全量解析时启用）



几条容易踩的结论

- sync_pdms 发布 DuckLake baseline，但**不写水位**：跑完全量同步系统仍然不「就绪」。
- 更进一步，**独立跑 sync_pdms 基本白费**——applied_sesno 仍为 0，会让 baseline_needs_full_parse 再解析一遍（幂等，只是白花时间）。正确做法是不手动开 total_sync，让基线路径自己驱动那次受限解析。
- SYS meta (SYST / DICT / GLB / GLOB) **也走基线**，不走 cold-start 重放：两条路用的不是同一个解析器，ADR-006 的跨块引用 (CURD / DBLS) 修复只落在 parse_pdms_db 上。
- 连带后果：UpdateScope 本身是从 SYS meta 库读出来的，所以 SYS 必须先建基线，DESI 才进得了执行范围——batch_worker 的 SCOPE_DIRTY 标志就是处理这件事的。
- readiness 不只是性能负担，它同时是「PE 已落、DuckLake 未提交」这个崩溃窗口的**唯一防线**，优化时只能换粒度、不能拿掉。